

Министерство образования и науки Республики Башкортостан
Государственное автономное профессиональное образовательное
учреждение
Уфимский колледж статистики, информатики и вычислительной техники

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____ 3.3. Курмашева
«_____» _____ 2026 г.

Курсовая работа
по дисциплине МДК 02.01 Технология разработки программного
обеспечения
на тему: «Разработка мобильного приложения для прослушивания
подкастов»
Специальность 09.02.07 «Обеспечение информационной безопасности
автоматизированных систем»

Руководитель курсовой работы
_____ О.В. Фатхулова
«_____» _____ 2026 г.

Студент гр. 23П-4
_____ И.Ф. Файзельгаянов
«_____» _____ 2026 г.

Уфа – 2026

Данная курсовая работа посвящена проектированию и разработке мобильного приложения для прослушивания подкастов с использованием современных кроссплатформенных технологий и асинхронного серверного REST API. В работе проанализированы основные принципы построения мобильных медиаплееров и систем дистрибуции потокового аудиоконтента. Описаны архитектурные преимущества и технические особенности выбранных программных средств.

Первая глава представляет характеристики и теоретические аспекты проектирования клиент-серверной информационной системы как средства структурирования и доставки медиаданных. Также эта глава поясняет обоснованность выбора реляционной СУБД MySQL в качестве хранилища данных, фреймворка FastAPI в качестве бэкенда и Flutter в качестве мобильной платформы с приведением подробных характеристик проектируемой базы данных.

Вторая глава описывает практическую разработку и программное внедрение серверного REST API в инфраструктуру приложения. Далее раскрывается эффективность реализации механизмов JWT-авторизации, валидации входящих .mp3 аудиофайлов по размеру и MIME-типу, а также проектирование адаптивного графического интерфейса мобильного клиента.

Рассмотрены принципы работы асинхронных сетевых запросов и управления состоянием приложения через паттерн Provider, а также проведен приемочный анализ работоспособности системы. В заключении работы предложены рекомендации по дальнейшему масштабированию хранилища данных и оптимизации битрейта аудиофайлов в зависимости от пропускной способности сети.

В заключении представлены основные выводы и практические результаты по проделанной работе.

Основными источниками информации для написания работы послужили труды отечественных и зарубежных авторов в области

проектирования баз данных, разработки мобильных систем и веб-служб, а также официальная техническая документация фреймворков FastAPI и Flutter.

Тема курсовой работы: «Разработка мобильного приложения для прослушивания подкастов»

Объём курсовой работы 35 страниц, на которых размещены 12 рисунков и 2 таблицы.

Ключевые слова: мобильное приложение, подкасты, Flutter, FastAPI, СУБД MySQL, REST API, управление состоянием, JWT-авторизация, потоковое аудио.

Объект исследования – процессы публикации, категоризации и воспроизведения аудиоконтента на мобильных устройствах.

Цель – проектирование и программная реализация клиент-серверного мобильного приложения для прослушивания и администрирования подкастов с применением кроссплатформенных программных средств.

В работе предложен анализ архитектурных решений для мобильного стриминга звука, рассмотрены способы физического хранения аудиофайлов на сервере, показаны механизмы разграничения прав доступа по ролям (Слушатель и Администратор).

Результат работы является показателем актуальности внедрения легковесных асинхронных API для раздачи медиаданных без использования дорогостоящих сторонних облачных платформ.

В курсовой работе на тему: «Разработка мобильного приложения для прослушивания подкастов» описаны программные компоненты серверной логики на Python и архитектура мобильного клиента во Flutter, приведены методы защиты пользовательских сессий посредством шифрования токенов JWT.

Выбраны оптимальные средства и пакеты для обеспечения бесперебойного аудиовещания в режиме реального времени на мобильных устройствах.

Курсовая работа посвящена разработке и внедрению информационной системы дистрибуции подкастов. В ней описаны сведения о структуре реляционных таблиц, о принципе работы асинхронного сервера, показана установка зависимостей, настройка сетевого взаимодействия между эмулятором Android и локальным хостом, а также развертывание мобильного клиента. Использование JWT-авторизации и хэширования паролей по алгоритму bcrypt обеспечивает сохранность персональных данных пользователей и предотвращает несанкционированный доступ к административным функциям управления контентом.

Человеческий фактор при вводе данных, попытки загрузки вредоносных или слишком объемных файлов — все это потенциальные риски стабильности системы, которые были устранены на этапе проектирования. Именно встроенная серверная валидация FastAPI и клиентские ограничения Flutter гарантируют полное отсутствие критических сбоев при некорректных действиях пользователей.

Кроссплатформенная разработка на базе Flutter и FastAPI — это эффективное решение для проектирования современных ориентированных на пользователя мобильных приложений.

Выпускная квалификационная работа на тему: «Исследование технологий блокчейн в борьбе с киберпреступностью»

Студент – Файзельгаянов Ильнур Фадисович.

Дипломный руководитель – О.В. Фатхулова.

Объем выпускной квалификационной работы 35 страниц, на которых размещены 12 рисунков и 2 таблицы.

Ключевые слова: мобильное приложение, подкасты, Flutter, FastAPI, СУБД MySQL, REST API, управление состоянием, JWT-авторизация, потоковое аудио.

Объектом исследования являются процессы публикации, категоризации и воспроизведения аудиоконтента на мобильных устройствах.

В курсовую работу входит содержание, введение, две главы (теоретическая и практическая), заключение и список использованных источников.

В введении раскрывается актуальность темы, ставятся цель и задачи исследования.

В аналитической части рассматриваются понятие клиент-серверной архитектуры, особенности потокового вещания аудиоданных и осуществляется проектирование структуры базы данных СУБД MySQL.

В практической части подробно рассматриваются программные инструменты реализации REST API на FastAPI и построение графического интерфейса мобильного клиента с плеером на Flutter.

Заключение содержит выводы по проделанной работе и достигнутым результатам.

СОДЕРЖАНИЕ

ЛИСТ

ВВЕДЕНИЕ.....	7
1. Общая часть	9
1.1. Анализ предметной области и обоснование актуальности разработки	9
1.2. Обоснование выбора архитектуры и технологического стека.....	10
1.3. Спецификация и структура реляционной базы данных	11
2. Специальная часть	12
2.1. Реализация серверного приложения.....	12
2.2. Разработка пользовательского интерфейса и логики.	14
2.2.1. Архитектура клиентских модулей.....	16
2.3. Протокол тестирования разработанной информационной системы.	17
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	23

ВВЕДЕНИЕ

Развитие современных беспроводных сетей передачи данных и повсеместное распространение смартфонов привели к качественному изменению способов потребления медиаконтента. Одним из наиболее динамично растущих сегментов цифрового вещания стали аудиоподкасты — регулярные разговорные или образовательные передачи, транслируемые через интернет. Специфика мобильного использования требует от подобных систем высокой оптимизации сетевого трафика, плавного воспроизведения потокового аудио в реальном времени и интуитивно понятного интерфейса, адаптированного для работы «на ходу».

Разработка проприетарного программного решения для прослушивания подкастов позволяет избежать ограничений крупных публичных платформ, таких как отсутствие гибкой категоризации, жесткие рамки монетизации и зависимость от внешних облачных инфраструктур. Использование клиент-серверной архитектуры с выделенным реляционным хранилищем и кроссплатформенным мобильным клиентом обеспечивает необходимый уровень масштабируемости, отказоустойчивости и автономии.

Объектом исследования является процесс публикации, структурирования и дистрибуции потокового аудиоконтента (подкастов) посредством мобильных систем.

Предметом исследования является проектирование и программная реализация клиент-серверного мобильного приложения для прослушивания подкастов на базе СУБД MySQL, серверного REST API на FastAPI и кроссплатформенного клиента на Flutter.

Целью курсовой является создание масштабируемой и производительной информационной системы, обеспечивающей надежную авторизацию пользователей, удобную навигацию и контекстный поиск по аудиокаталогу, а также бесперебойное воспроизведение подкастов на мобильных устройствах.

Для достижения цели работы были определены следующие задачи:

- изучить теоретические аспекты проектирования клиент-серверных архитектур мобильных приложений и потокового вещания медиаданных;
- разработать реляционную модель базы данных СУБД MySQL для учета учетных записей и аудиофайлов;
- спроектировать и разработать серверную часть информационной системы (REST API) на базе асинхронного веб-фреймворка FastAPI;
- реализовать клиентскую часть мобильного приложения на фреймворке Flutter с использованием паттерна управления состоянием Provider;
- провести функциональное тестирование разработанной системы для подтверждения стабильности её работы при граничных нагрузках и вводе некорректных данных.

1. Общая часть

1.1. Анализ предметной области и обоснование актуальности разработки

Современные системы хранения и дистрибуции подкастов представляют собой сложные программные комплексы, работающие по модели «клиент-сервер». Основная задача таких систем — предоставить пользователю быстрый доступ к аудиофайлам без необходимости их предварительного скачивания целиком на устройство, что реализуется с помощью технологии прогрессивного скачивания (progressive download) или потокового вещания (streaming).

С точки зрения контент-менеджмента, предметная область характеризуется наличием двух основных ролей:

- слушатель (Client/Listener): регистрируется в системе, осуществляет поиск подкастов по ключевым словам, фильтрует каталог по категориям и прослушивает аудиозаписи с помощью интерактивного плеера;
- администратор (Admin): осуществляет глобальное администрирование системы, включая просмотр пользователей, добавление новых подкастов (с метаданными и аудиофайлами) и удаление устаревшего контента.

Актуальность разработки собственного приложения обусловлена необходимостью локального хранения медиаресурсов на собственных мощностях предприятия, гибкого управления базой пользователей и исключения рисков несанкционированного доступа к служебным или образовательным аудиоматериалам организации.

1.2. Обоснование выбора архитектуры и технологического стека.

Для реализации проекта была выбрана классическая трехзвенная клиент-серверная архитектура, включающая в себя уровень представления (мобильный клиент), уровень бизнес-логики (REST API сервер) и уровень хранения данных (СУБД).

В качестве серверного фреймворка выбран FastAPI. FastAPI является одним из наиболее производительных веб-фреймворков на Python благодаря интеграции библиотек Starlette и Pydantic. Использование стандарта асинхронного программирования ASGI (uvicorn) позволяет серверу эффективно обрабатывать тысячи параллельных запросов ввода-вывода (I/O-bound), что критически важно при раздаче аудиофайлов. Для взаимодействия с СУБД используется SQLAlchemy ORM, автоматизирующая построение безопасных SQL-запросов.

Для разработки мобильного интерфейса выбран кроссплатформенный фреймворк Flutter. Использование компилируемого языка Dart обеспечивает высокую производительность графики за счет собственного движка отрисовки Impeller. Flutter позволяет использовать единую кодовую базу для сборки под платформы Android и iOS. Управление состоянием (State Management) реализовано через пакет Provider, разделяющий визуальную верстку и логику работы с данными.

Для хранения структурированной информации выбрана реляционная СУБД MySQL. Она обеспечивает высокую скорость чтения транзакций, надежность хранения данных и полную совместимость со стандартом SQL.

1.3. Спецификация и структура реляционной базы данных

Схема базы данных спроектирована с учетом обеспечения целостности и минимизации избыточности данных. Ниже представлены таблицы 1.3.1 – 1.3.2 физической структуры базы данных MySQL.

Таблица 1.3.1 – структура таблицы users (учетные записи пользователей)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	id	PRIMARY KEY	первичный, авто-инкремент
Электронная почта пользователя	email	VARCHAR(150)	уникальное, не пустое
Хэш-код пароля	hashed_pass word	VARCHAR(255)	не пустое
Роль доступа	role	VARCHAR(50)	стандартное значение listener

Таблица 1.3.2 – структура таблицы podcasts (каталог подкастов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор подкаста	id	PRIMARY KEY	первичный, авто-инкремент
Название аудиозаписи	title	VARCHAR(255)	не пустое
Категория подкаста	category	VARCHAR(100)	не пустое
Описание содержания подкаста	description	VARCHAR(1000)	не пустое
Путь к аудиозаписи на сервере	file_path	VARCHAR(500)	не пустое
Дата и время публикации	created_at	DATETIME	стандартное, текущее время с датой

2. Специальная часть

2.1. Реализация серверного приложения

Серверная часть функционирует как REST API, обмениваясь с мобильным клиентом данными в формате JSON. Безопасность эндпоинтов реализована на базе авторизации по протоколу OAuth2 с использованием Bearer JWT-токенов подписи HS256.

Для хэширования паролей на стороне бэкенда используется библиотека `bcrypt`, исключающая хранение паролей пользователей в чистом виде.

Часть разработанных API-эндпоинтов серверной части представлена на рисунках 2.1.1 – 2.1.2:

```
@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def register(user_data: UserCreate, db: Session = Depends(get_db)):
    # Проверяем, существует ли уже пользователь с таким email.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if db_user:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Пользователь с таким email уже зарегистрирован"
        )

    # Создаем нового пользователя.
    hashed_pw = hash_password(user_data.password)
    new_user = User(
        email=user_data.email,
        hashed_password=hashed_pw,
        role="listener" # По умолчанию регистрируем с ролью Слушателя.
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

@router.post("/login", response_model=Token)
def login(user_data: UserLogin, db: Session = Depends(get_db)):
    # Поиск пользователя.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if not db_user or not verify_password(user_data.password, db_user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неверный email или пароль"
        )

    # Генерация JWT.
    access_token = create_access_token(data={"sub": db_user.email, "role": db_user.role})
    return Token(access_token=access_token, role=db_user.role)

@router.get("/users", response_model=List[UserResponse])
def list_users(
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    """Возвращает список всех зарегистрированных пользователей (только для Администратора)."""
    return db.query(User).all()
```

Рисунок 2.1.1 – часть разработанных API-эндпоинтов серверной части

```

# Публикация подкаста (только для роли `Администратора`).
@router.post("/", response_model=PodcastResponse, status_code=status.HTTP_201_CREATED)
async def create_podcast(
    title: str = Form(...),
    category: str = Form(...),
    description: str = Form(None),
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    # Проверка расширения файла.
    if not file.filename.lower().endswith(".mp3"):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Разрешены только файлы формата .mp3"
        )

    # Проверка MIME-типа.
    if file.content_type != "audio/mpeg":
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Неверный MIME-тип. Разрешен только audio/mpeg"
        )

    # Валидация размера файла (до 100 МБ).
    if file.size is not None:
        file_size = file.size
    else:
        # Резервный способ через прямое обращение к внутреннему файловому объекту.
        file.file.seek(0, 2)
        file_size = file.file.tell()
        file.file.seek(0)

    if file_size > 100 * 1024 * 1024:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Размер файла превышает лимит в 100 МБ"
        )

```

Рисунок 2.1.2 – часть разработанных API-эндпоинтов серверной части

2.2. Разработка пользовательского интерфейса и логики.

Интерфейс мобильного приложения разработан в соответствии с гайдлайнами Material Design 3 и ориентирован на темную цветовую палитру (Dark Premium) для снижения нагрузки на зрение при длительном использовании.

Проектирование функциональных возможностей мобильного приложения базируется на принципе разделения прав доступа между двумя категориями пользователей: Слушателем и Администратором. Для формализации требований к системе и описания сценариев взаимодействия пользователей с интерфейсом была разработана диаграмма прецедентов (Use Case Diagram).

Данная диаграмма позволяет на концептуальном уровне визуализировать иерархию функций приложения, выделив ключевые действия, доступные каждому из акторов. Сценарии использования включают в себя как базовые операции по работе с контентом (просмотр каталога, воспроизведение подкастов), так и административные функции по управлению базой данных и публикацией новых медиафайлов.

На рисунке 2.2.1 представлена Use Case диаграмма.



Рисунок 2.2.1 – Use Case диаграмма

2.2.1. Архитектура клиентских модулей.

Логика приложения разделена на три ключевых провайдера:

- AuthProvider: сохраняет JWT-токен во внутреннем хранилище устройства через shared_preferences и управляет сессией пользователя;
- PodcastProvider: запрашивает списки подкастов у API сервера, обрабатывает контекстный поиск и горизонтальную фильтрацию по чипсам категорий;
- PlaybackProvider: инкапсулирует работу с нативным аудиоплеером смартфона посредством пакета audioplayers. Отслеживает текущую позицию вещания, буферизацию, общую длительность трека и статус воспроизведения.

Интерфейс плеера (BottomSheet) - для экрана воспроизведения реализована интерактивная выдвигаемая панель. Чтобы избежать конфликтов переполнения разметки на этапе изменения размеров панели в момент анимации раскрытия (от 84px до 275px).

2.3. Протокол тестирования разработанной информационной системы.

В ходе приемочного тестирования системы были проверены все критические сценарии использования программы.

В таблицах 2.3.1 – 2.3.4 представлены протоколы тестирования.

Таблица 2.3.1 – протокол тестирования успешной регистрации слушателя.

Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при регистрации нового слушателя
Этапы теста	Открыть форму регистрации; заполнить поля «Email», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»
Тестовые данные	Email: testingList@gmail.com , Пароль: 7878488Aa, Повторный пароль: 7878488Aa
Ожидаемый результат	Учётная запись создана
Фактический результат	Учётная запись создана. Результат представлен на рисунке 2.3.1.1



Рисунок 2.3.1.1 – фактический результат

Таблица 2.3.2 – протокол тестирования регистрации слушателя с уже занятой почтой.

Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытке регистрации нового слушателя с уже существующей почтой
Этапы теста	Открыть форму регистрации; заполнить поля «Email (уже существующего слушателя)», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»
Тестовые данные	Email: testingList@gmail.com , Пароль: 7878488Аа, Повторный пароль: 7878488Аа
Ожидаемый результат	Ошибка, учетная запись не создана
Фактический результат	Ошибка, учетная запись не создана. Результат представлен на рисунке 2.3.2.1



Рисунок 2.3.2.1 – фактический результат

Таблица 2.3.3 – протокол тестирования успешной аутентификации.

Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование аутентификации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при аутентификации пользователя
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: testingList@gmail.com , Пароль: 7878488Aa
Ожидаемый результат	Успешная аутентификация, получение JWT-токена
Фактический результат	Успешная аутентификация, получение JWT-токена. Результат представлен на рисунке 2.3.3.1

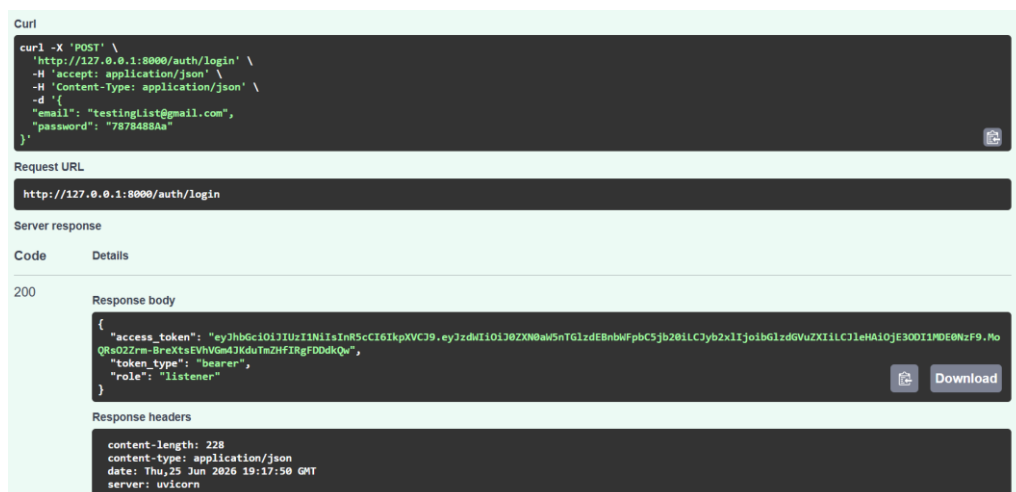


Рисунок 2.3.3.1 – фактический результат

Таблица 2.3.4 – протокол тестирования аутентификации с неверными данными.

Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование аутентификации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытке аутентификации пользователя с некорректными данными
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: aa@gsd , Пароль: -
Ожидаемый результат	Ошибка, неудачная аутентификация
Фактический результат	Ошибка, неудачная аутентификация. Результат представлен на рисунке 2.3.4.1



Рисунок 2.3.4.1 – фактический результат

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была полностью спроектирована и реализована клиент-серверная информационная система для публикации и прослушивания подкастов.

В процессе выполнения проекта были достигнуты следующие результаты:

- Разработана реляционная физическая схема базы данных на СУБД MySQL, обеспечивающая хранение данных учетных записей пользователей и атрибутов подкастов без избыточности;
- Создано серверное REST API на базе FastAPI (Python) с асинхронным циклом обработки запросов (uvicorn), безопасным односторонним хэшированием паролей алгоритмом bcrypt и JWT-авторизацией. Реализованы строгие серверные правила проверки размера и расширения загружаемых аудиоматериалов;
- Разработано кроссплатформенное мобильное приложение на базе фреймворка Flutter (Dart) с использованием паттерна управления состоянием Provider;
- Спроектирован современный и адаптивный пользовательский интерфейс в темном стиле, удобным для длительного взаимодействия.

Все задачи курсовой работы решены в полном объеме, цель работы успешно достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Издания

1. «Современный стиль программирования» (2021, Питер) – Автор Б. Любанович.
2. «Flutter в действии» (2020, Москва) – Автор Х. Кингсли.
3. «Базы данных: учебник» (2021, Москва) – Автор Кумскова И.А.

Электронные ресурсы:

1. fastapi.tianglo.com
2. docs.flutter.dev